Quantization

Why Large Models Are a Problem?

Models such as BERT and LLaMA are considered very large-scale neural networks, containing hundreds of millions or even billions of parameters. While these models achieve state-of-the-art performance in many natural language processing tasks, their large size creates several challenges. They require high memory consumption, powerful GPUs, expensive computational resources, and they are difficult to deploy on edge devices such as mobile phones or laptops. For example, BERT-base contains around 110 million parameters and takes about 420 MB in FP32 precision, while LLaMA-7B contains 7 billion parameters and can require up to 28 GB of memory when stored in full precision. To address this issue, one of the most widely used solutions is quantization.

What is Quantization?

Quantization is a technique that reduces the numerical precision of model weights and activations. Instead of using 32-bit floating point representation (FP32), quantization stores parameters in lower-bit formats such as FP16, INT8, or even INT4. This significantly reduces the memory footprint of the model and improves inference speed. For instance, FP16 reduces model size by half, INT8 reduces it by four times, and INT4 can reduce it by up to eight times compared to FP32. This allows large models to run efficiently on limited hardware.

Types of Quantization

There are two main types of quantization. The first is Post-Training Quantization (PTQ), where the model is quantized after training without needing retraining. This approach is simple and fast but may introduce a small accuracy drop. The second is Quantization-Aware Training (QAT), where quantization is applied during training so the model learns to adapt to reduced precision, leading to better accuracy but requiring additional training effort.

Quantization can be applied easily using modern deep learning frameworks. For example, dynamic INT8 quantization can be applied to BERT using PyTorch as follows:

```
from transformers import AutoModelForSequenceClassification

import torch

from torch.quantization import quantize_dynamic


model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased")
```

```
quantized_model = quantize_dynamic(
    model,
    {torch.nn.Linear},
    dtype=torch.qint8
)

print("BERT model successfully quantized to INT8")
```

This reduces model size and improves CPU inference performance with minimal accuracy loss.

For very large models like LLaMA, more aggressive quantization such as 4-bit quantization is commonly used. Using the bitsandbytes library with Hugging Face Transformers, LLaMA can be loaded in 4-bit precision as follows:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "meta-llama/Llama-2-7b-hf"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    load_in_4bit=True,
    device_map="auto"
)

print("LLaMA loaded successfully in 4-bit quantized form")
```

This makes it possible to run LLaMA models on GPUs with only 4–6 GB of memory instead of requiring tens of gigabytes.

Quantization is also used in advanced fine-tuning methods such as QLoRA (Quantized Low-Rank Adaptation). In QLoRA, the base model is stored in 4-bit precision while only small adapter layers are trained, allowing efficient fine-tuning with low memory usage. Example:

```
from peft import LoraConfig, get_peft_model


config = LoraConfig(
   r=8,
   lora_alpha=16,
   target_modules=["q_proj", "v_proj"]
)


model = get_peft_model(model, config)


print("QLoRA fine-tuning enabled")
```

Conclusion

Overall, quantization is one of the most effective techniques to address the problem of large model size. It reduces memory requirements, speeds up inference, lowers deployment costs, and enables large language models to be used on consumer hardware and edge devices. Although it may slightly reduce accuracy in some cases, quantization remains a key solution for practical deployment of models such as BERT and LLaMA.